Since 1983

Dr. Vishwanath Karad

MIT WORLD PEACE UNIVERSITY | PUNE

TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

**Department of Computer Engineering and Technology**

# MINI PROJECT REPORT

## Computer Networks Laboratory

**(CSE20110)**

Project Title

## DISTRIBUTED CHAT WITH LEADER ELECTION

Github Link - https://github.com/mitansh-dekate/CN-Lab/tree/main/mini-project

**SUBMITTED BY**


| PRN | Student Name |
|---|---|
| 1262241930 | Mitansh Dekate |
| 1262243181 | Kartik Saxena |
| 1262243385 | Om Mali |
| 1262241292 | Harpal Kalankar |


**Group 11, Panel K (iOS Batch)**

**Third Year B.Tech. Computer Engineering and Technology**

**Academic Year 2026–2027**

**UNDER THE GUIDANCE OF**

**Dr. Uma Pujeri**

**Dr. Swati**

Since 1983

Dr. Vishwanath Karad

MIT WORLD PEACE UNIVERSITY | PUNE

TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

**Department of Computer Engineering and Technology**

# CERTIFICATE

This is to certify that the mini project report entitled **"Distributed Chat with Leader Election"** has been carried out by the following students of Third Year B.Tech. Computer Engineering and Technology, Panel K (iOS Batch), Group No. **11**, under my guidance during the Academic Year 2026–2027, in partial fulfilment of the requirements of the Computer Networks Laboratory (CSE20110), Laboratory Continuous Assessment 3.

| Sr. No. | PRN | Student Name |
|---|---|---|
| 1 | 1262241930 | Mitansh Dekate |
| 2 | 1262243181 | Kartik Saxena |
| 3 | 1262243385 | Om Mali |
| 4 | 1262241292 | Harpal Kalankar |

To the best of my knowledge, the work reported in this report is the original work of the students and has not been submitted elsewhere for the award of any other credit.

Date: 25 October 2026
Place: Pune

Dr. Uma Pujeri
Project Guide / Course Faculty
Computer Networks Laboratory

# ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our project guide and course faculties, **Dr. Uma Pujeri & Dr. Swati**, for the guidance, feedback and encouragement that shaped this mini project from the first proposal to the final report.

We thank the Department of Computer Engineering and Technology for providing the Computer Networks Laboratory course framework, the evaluation criteria and the opportunity to design, build and test a networked system on our own.

We also acknowledge the authors of the textbooks, research papers and documentation listed in the References chapter. Their explanations of failure detection, leader election and replication were the foundation on which this project was designed.

Finally, we thank our teammates, friends and families for their support during the development and testing of the project.

# ABSTRACT

A single chat server is a single point of failure: when it crashes, every connected user loses the service. This project designs and implements a distributed chat service made of several server nodes that keep working when their coordinating leader fails. The nodes are written in Python 3 using only the standard library and communicate over TCP with newline-delimited JSON messages. Every node sends heartbeats every 0.5 s and presumes a peer dead after 1.5 s of silence. When the leader is lost, the survivors elect a new one by majority voting with randomised timeouts and one vote per term; Bully and Ring (Chang–Roberts) elections were also implemented for comparison.

The elected leader gives every chat message a sequence number, replicates it to the followers and commits it only when a majority holds it, so all clients see one identical order. Restarted nodes resynchronise their history, and clients reconnect to another node and retry without creating duplicates. A web dashboard injects crashes, network partitions, delay and packet loss. Scenario tests on a five-node cluster passed for all three algorithms with no message lost, duplicated or reordered. In benchmarks, recovery after a leader crash took about 2.1 s for voting and about 1.5 s for Bully and Ring, while voting used only $3n-4$ election messages for n nodes.

**Keywords:** distributed systems, leader election, heartbeat failure detection, replicated log, TCP sockets, fault tolerance, Python.

# TABLE OF CONTENTS


| Chapter No. | Chapter Name | Page No. |
|---|---|---|
| 1 | Introduction and Project Objectives<br>• 1.1 Introduction and Background<br>• 1.2 Problem Statement<br>• 1.3 Scope and Assumptions<br>• 1.4 Objectives | 1 |
| 2 | Proposed Design and Requirements<br>• Architecture and Network Topology<br>• Tools and System Requirements<br>• Addressing and Protocol Selection | 3 |
| 3 | Implementation and Technical Working<br>• 3.1 Methodology<br>• 3.2 Configuration and Code Implementation<br>• 3.3 Protocols and Technical Working<br>• 3.4 Execution Procedure | 6 |
| 4 | Testing and Results<br>• 4.1 Test Plan<br>• 4.2 Results and Execution Evidence<br>• 4.3 Objective Achievement | 12 |
| 5 | Discussion and Conclusion<br>• 5.1 Discussion and Comparative Analysis<br>• 5.2 Advantages and Applications<br>• 5.3 Limitations<br>• 5.4 Challenges and Troubleshooting<br>• 5.5 Future Scope<br>• 5.6 Conclusion | 18 |
| | References | 21 |

# LIST OF FIGURES


| Figure Name | Page No. |
|---|---|
| Proposed architecture and network topology of the five-node cluster | 3 |
| Flowchart of the voting election run by every node | 8 |
| Message sequence of an election after the leader (Node 5) crashes | 9 |
| Message sequence for one chat message (write path) | 10 |
| Dashboard showing a healthy five-node cluster with Node 1 as leader | 11 |
| Election in progress after a forced re-election (four candidates compete in term 2 | 14 |
| Election after the leader (Node 3) was crashed during message traffic | 14 |
| Cluster after recovery: Node 4 leads term 5 and all live nodes hold identical logs | 15 |
| Winners of repeated elections for voting and Bully | 15 |
| Election messages and recovery time against cluster size (benchmark) | 16 |

# LIST OF TABLES


| Table Title | Page No. |
|---|---|
| Project objectives and verification methods | 2 |
| Source files and their roles | 3 |
| Tools and system requirements | 4 |
| Addressing and ports (default localhost layout) | 5 |
| Timing constants used by every node | 5 |
| Message types | 7 |
| Test plan and results | 12 |
| Mean benchmark results after a leader crash (3 trials per row) | 16 |
| Objective achievement | 17 |
| Comparison of the three election algorithms (measured values) | 18 |
| Issues, causes and solutions | 19 |

# CHAPTER 1 : INTRODUCTION AND PROJECT OBJECTIVES

## 1.1 Introduction and Background

A chat service is a client–server application: clients open connections to a server, which receives every message and forwards it to the other participants. If one server holds all the state, its failure stops the service for everybody. The usual remedy is to run several server replicas, but the replicas then have to agree on who is in charge, in which order the messages happened and which messages are safely stored. These are classic problems of distributed systems, studied under the headings of failure detection, leader election, replication and logical time.

In an asynchronous network a crashed machine cannot be told apart from a slow one. Practical systems therefore use heartbeats and timeouts: a peer that stays silent for longer than a timeout is suspected to have failed. The timeout is a trade-off between detecting real failures quickly and wrongly suspecting a healthy but slow peer. The nodes in this project talk over the Transmission Control Protocol, which gives reliable, ordered byte streams between two endpoints but no guarantee about delay.

Three well-known election approaches are relevant. In the Bully algorithm of Garcia-Molina, a node that notices a failure challenges every node with a higher identifier and the highest live identifier wins; it is simple and deterministic, but it always favours one machine and its message count can grow quickly when several nodes start elections together. The ring algorithm of Chang and Roberts circulates the largest identifier seen around a logical ring and needs about n log n messages on average. The Raft consensus algorithm of Ongaro and Ousterhout uses numbered terms, one vote per node per term and randomised election timeouts so that nodes rarely compete at the same moment; its leader then replicates a log and commits entries once a majority stores them. The voting election in this project is a simplified version of that idea.

Two further ideas are used. Lamport’s logical clocks give events a consistent ordering without synchronised physical clocks. The CAP result formalised by Gilbert and Lynch states that during a network partition a service must choose between consistency and availability; this project chooses consistency, so a minority partition stops committing messages instead of diverging.

### 1.2 Problem Statement

Design and implement a chat service built from several server nodes such that (a) the failure of the coordinating leader is detected automatically, (b) the surviving nodes agree on exactly one new leader without human intervention, (c) no message accepted by the service is lost, duplicated or reordered across the failover, (d) a crashed node can restart and recover the full history, and (e) network faults such as partitions, delay and packet loss do not produce inconsistent histories. In addition, the project should compare a randomised voting election with the classical Bully and Ring elections in terms of messages sent and time to recover.

### 1.3 Scope and Assumptions

The system is a teaching-scale prototype. Users chat through a terminal client or through a browser dashboard. The cluster consists of N server nodes (five by default; three to nine were tested), each an operating-system process, connected in a full TCP mesh. In the evaluation all nodes run on one computer and talk over the loopback interface; the configuration also supports nodes on several computers, but that setup was not evaluated for this report.

The design assumes crash-stop failures (a node either works correctly or stops; it never sends wrong data), a fixed set of nodes known at start-up, and a trusted network. Message history is kept in memory only. The service makes progress only while a majority of the nodes can communicate. Persistence, encryption, authentication and dynamic membership are outside the scope.

### 1.4 Objectives

**Table 1.1 Project objectives and verification methods**

| ID | Objective | Verification | Result in |
|---|---|---|---|
| O1 | Detect leader failure with heartbeats (interval 0.5 s, timeout 1.5 s) | Scenario test 3; benchmark detection time | Sec. 4.2 |
| O2 | Elect exactly one leader by majority voting with randomised timeouts | Scenario tests 1 and 5; winner-distribution test | Sec. 4.2 |
| O3 | Deliver chat messages in one total order, with no loss or duplication | Scenario tests 2 and 3 | Sec. 4.1 |
| O4 | Replicate with majority commit; let restarted nodes recover the full log | Scenario test 4 | Sec. 4.1 |
| O5 | Stay consistent under a network partition and converge after it heals | Scenario test 5 | Sec. 4.1 |
| O6 | Let clients fail over to another node and retry safely | Scenario test 3 (clients keep typing) | Sec. 4.1 |
| O7 | Compare voting, Bully and Ring on messages and recovery time | Benchmark, 3 algorithms × 4 cluster sizes × 3 trials | Sec. 4.2 |
| O8 | Provide a live dashboard with fault injection | Demonstration screenshots | Sec. 4.2 |

# CHAPTER 2 : PROPOSED DESIGN AND REQUIREMENTS

## 2.1 Architecture and Network Topology

Figure 2.1 shows the proposed architecture. Every server node is an independent process that listens on one TCP port for both peer nodes and chat clients, and on a second HTTP port that reports its status and accepts control commands. Each node opens a TCP connection to every other node, giving a full mesh. A client may connect to any node. A node that is not the leader forwards the client's message to the leader, so clients never need to know who the leader is.

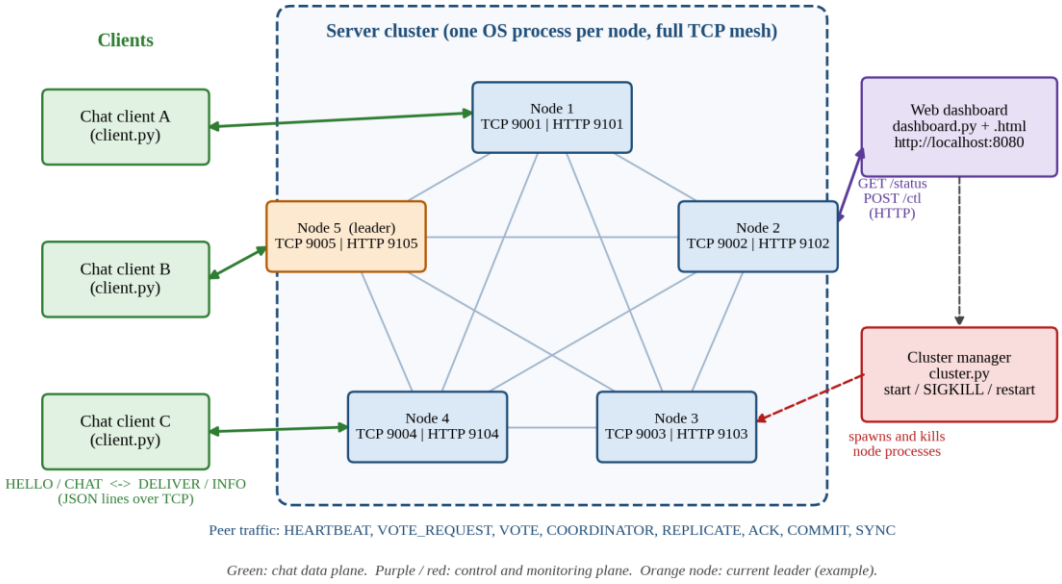



**Figure 2.1 Proposed architecture and network topology of the five-node cluster**

The data path of a chat message is: client, its node, the leader, replication to the followers, commit once a majority stores the message, and delivery to every client from the node it is attached to. The control path is separate: the dashboard polls each node's HTTP status endpoint and sends commands to it, and the cluster manager starts, kills and restarts node processes. Table 2.1 lists the source files and their roles.

**Table 2.1 Source files and their roles**

| File | Role |
|---|---|
| `node.py` | Server node: peer links, heartbeats, three election algorithms, replicated log, client handling, HTTP status/control |
| `protocol.py` | Wire format (JSON lines), Lamport clock, outbound peer link with a send queue |
| `config.py` | Ports, timing constants and cluster layout |
| `client.py` | Terminal chat client with automatic reconnection and resend |
| `cluster.py` | Starts, kills and restarts node processes on one computer |
| `dashboard.py` + `dashboard.html` | Web dashboard: cluster view, live election, chat, failure injection |

| File | Role |
|---|---|
| `tests/*.py`, `benchmark.py` | Scenario test (crash, restart, partition), winner-distribution test and benchmark of all three algorithms |

### 2.2 Tools and System Requirements

The implementation uses only the Python standard library for the cluster itself: `socket` for TCP, `threading` for concurrency, `json` for message encoding and `http.server` for the status interface. This keeps the system easy to install and to read. Table 2.2 lists the tools used.

**Table 2.2 Tools and system requirements**

| Tool / resource | Version / specification | Purpose |
|---|---|---|
| Python | Python 3.14 | Implementation language (standard library only) |
| Operating system | Windows / MacOS | Runs the node processes |
| Hardware | One computer, loopback network (127.0.0.1) | Hosts all five nodes, the clients and the dashboard |
| Web browser | Any modern browser | Shows the dashboard at port 8080 |
| Matplotlib | Python Library | Draws the benchmark graph |

The functional requirements derived from the problem statement are:

- detect a silent leader within about 1.5 s and start an election automatically;
- elect one leader that a majority of the nodes agree on, using one of three selectable algorithms;
- assign every chat message one sequence number and deliver messages in that order only after a majority has stored them;
- let a restarted or reconnected node catch up with the committed history;
- let clients move to another node and resend unacknowledged messages without duplicates; and
- allow faults (crash, isolation, delay, packet loss) to be injected and observed live.

The non-functional requirements are consistency before availability, no third-party dependencies for the cluster, and enough logging and status information to explain every election afterwards.

### 2.3 Addressing and Protocol Selection

Table 2.3 shows the default address layout. Node number k uses TCP port 9000 + k for peers and clients and HTTP port 9100 + k for status and control. All addresses are on the loopback interface, so no gateway is involved.

**Table 2.3 Addressing and ports (default localhost layout)**

| Device / interface | Address : port | Gateway | Role |
|---|---|---|---|
| Node k, TCP | 127.0.0.1 : 9001–9005 | N/A (loopback) | Peer traffic and client connections |
| Node k, HTTP | 127.0.0.1 : 9101–9105 | N/A (loopback) | Status and control endpoint |
| Dashboard | 127.0.0.1 : 8080 | N/A (loopback) | Web user interface |
| Chat client | Ephemeral port to a node | N/A (loopback) | Terminal chat client |
| Several computers | Addresses in a cluster file | LAN gateway | Supported by configuration, not evaluated here |

**TCP** was chosen because the replication logic needs reliable, in-order delivery between two nodes; UDP would have required reimplementing acknowledgements and ordering. Messages are single JSON objects terminated by a newline, which makes the stream easy to split, easy to read in logs and easy to extend. The dashboard uses plain HTTP because browsers cannot open raw TCP sockets.

All timing behaviour is controlled by a small set of constants, shown in Table 2.4. The failure timeout is three heartbeat intervals, so a single lost or delayed heartbeat does not trigger an election, while a real crash is noticed within about 1.5 s.

**Table 2.4 Timing constants used by every node**

| Constant | Value | Meaning |
|---|---|---|
| `HEARTBEAT_INTERVAL` | 0.5 s | How often a node sends a heartbeat to each peer |
| `FAILURE_TIMEOUT` | 1.5 s | Silence after which a peer is presumed dead |
| `STARTUP_GRACE` | 1.5 s | Initial period before a missing leader counts as a failure |
| `VOTE_TIMEOUT_MIN` / `MAX` | 0.5 s / 1.5 s | Range of the random wait before a node becomes a candidate |
| `CATCHUP_WAIT` | 1.0 s | Time a new leader waits for peers' logs before serving |
| `CLIENT_RESEND_AFTER` | 4.0 s | Client resends a message that has not been delivered |

# CHAPTER 3 : IMPLEMENTATION AND TECHNICAL WORKING

## 3.1 Methodology

The system is organised in layers, each depending only on the layer below it. The lowest layer is the wire format and the peer links (`protocol.py`). On top of it sit heartbeat failure detection, the three election algorithms, the leader-ordered replicated log, and the client interface, all in `node.py`. The dashboard, cluster manager, scenario tests and benchmark use only the nodes' public interfaces: the TCP port for clients and the HTTP status and control port.

Every node runs the same program. A node is always in one of three states: **follower** (it accepts a leader), **election** (a leader is being chosen) or **leader** (it orders and replicates messages). Several threads run in each node: one accepts connections, one reads each connection, one sends heartbeats, and a monitor thread checks every 0.2 s whether the leader has gone silent. A single lock protects the node's state.

## 3.2 Configuration and Code Implementation

This section shows the three pieces of code that carry the core ideas; the complete source is in Appendix A. Every message is one JSON object followed by a newline (the `encode` function in `protocol.py`).

**Failure detection.** The monitor thread compares the time since the leader's last heartbeat with `FAILURE_TIMEOUT`. If the leader has been silent for longer, the follower forgets it and starts an election (Listing 1).

```
if state == FOLLOWER:
    if leader is not None and now - self.last_seen[leader] > cfg.FAILURE_TIMEOUT:
        self.log_event(f"leader N{leader} stopped sending heartbeats -> presumed DEAD",
"failure")
        with self.lock:
            self.leader_id = None
        self.start_election(f"leader N{leader} failed")
```

*Listing 1: leader failure detection in the monitor thread (node.py).*

**Granting a vote.** A node answers a vote request positively only when all safety conditions hold (Listing 2): no live leader is known, the term is newer than any term in which it has voted or seen a leader, and the candidate's log is at least as complete as its own.

```
reason = None
if term == self.vote_term and self.voted_for == cand:
    pass                                  # repeated request: same answer
elif leader_alive:
    reason = "a leader is still alive"
elif term <= self.vote_term:
    reason = f"already voted in term {self.vote_term}"
elif term <= self.max_lead_term:
    reason = "term is too old"
elif (m["last_term"], m["last_seq"]) < self._last_info():
    reason = "candidate's log is behind mine"
granted = reason is None
```

*Listing 2: vote-granting rules (node.py, on_vote_request).*

**Committing a message.** The leader advances the commit index only while a majority of nodes (including itself) acknowledge the next sequence number (Listing 3). Only committed messages are delivered to clients.

```
while self.commit_index < len(self.log):
    s = self.commit_index + 1
    if len(self.acks.get(s, {self.id})) >= self.majority:
        self.commit_index = s
        self.acks.pop(s, None)
        advanced = True
    else:
        break
```

*Listing 3: majority commit rule (node.py, _try_commit).*

### 3.3 Protocols and Technical Working

#### *3.3.1 Message types*

Table 3.1 lists the messages exchanged. Every message sent between nodes also carries the sender's identifier and a Lamport timestamp; an illustrative vote request looks like this:

```
{"type":"VOTE_REQUEST","term":6,"last_term":5,"last_seq":13,"from":2,"lc":412}
```

**Table 3.1 Message types**

| Message | Between | Purpose |
|---|---|---|
| `HEARTBEAT` | Node ↔ node | Liveness plus state, leader, term and log position, every 0.5 s |
| `VOTE_REQUEST`, `VOTE` | Candidate ↔ node | Voting election: ask for, and grant or refuse, a vote in a term |
| `ELECTION`, `ELECTION_OK` | Node ↔ node | Bully election: challenge higher ids; "stand down" reply |
| `RING_ELECTION` | Node → next node | Ring election: token carrying the best candidate id |
| `COORDINATOR` | Winner → all | Announces the new leader and its term |
| `RESIGN` | Leader → all | Leader steps down on request (used to force an election) |
| `LOG_QUERY`, `LOG_REPLY` | New leader ↔ node | Collects peers' logs before the leader serves writes |
| `FORWARD` | Follower → leader | Hands a client's message to the leader |
| `REPLICATE`, `ACK` | Leader ↔ follower | Append a log entry; follower reports how far its log reaches |

| Message | Between | Purpose |
|---|---|---|
| `COMMIT` | Leader → follower | Entries up to this number are safe to deliver |
| `SYNC_REQ`, `SYNC_RESP` | Follower ↔ leader | Repair a gap or a divergent suffix in a follower's log |
| `HELLO`, `CHAT` | Client → node | Introduce the client and where it stopped reading; send a message |
| `DELIVER`, `INFO` | Node → client | A committed entry in total order; periodic status for failure detection |

### *3.3.2 Heartbeat failure detection*

Every node sends a heartbeat to every peer every 0.5 s. A peer is presumed dead when nothing has been heard from it for 1.5 s. Because the monitor thread polls every 0.2 s, a crash is normally noticed between about 1.0 s and 1.7 s after it happens, which matches the measured detection times in Chapter 4. A follower reacts only to the silence of its leader; the silence of an ordinary peer merely removes that peer from the set used to count quorum.

### *3.3.3 Voting election*

The voting election, shown in Figure 3.1, works as follows. When a node notices that the leader has failed, it enters the election state and draws a random timeout between 0.5 s and 1.5 s. If a vote request from another node arrives first, the node acts as a voter. If its own timer expires first, it becomes a candidate: it raises the term number above every term it has seen, votes for itself and broadcasts `VOTE_REQUEST` with its term and the position of its log.

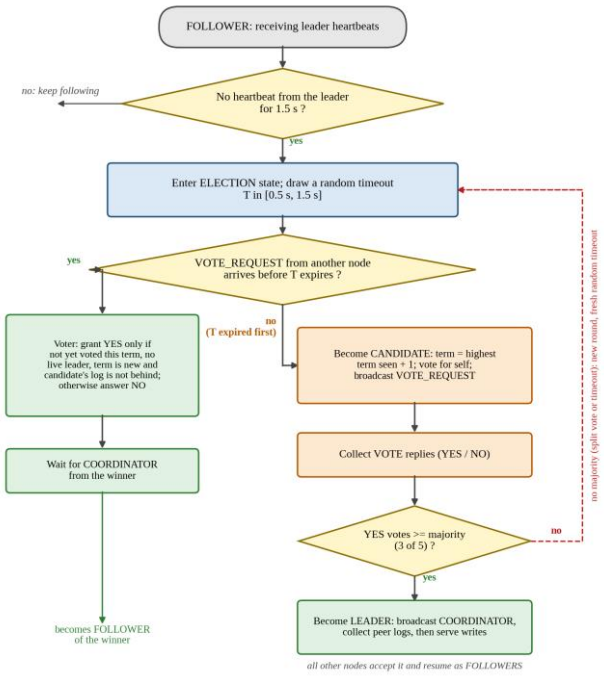



**Figure 3.1 Flowchart of the voting election run by every node**

A voter grants at most one vote per term and applies the rules in Listing 2. A candidate that collects votes from a majority (3 of 5 nodes) becomes leader and broadcasts `COORDINATOR`; every other node then accepts it as the leader of that term. If several nodes become candidates at nearly the same moment, the votes split and nobody reaches a majority. The round then times out and every node draws a new random timeout, so the next attempt is very unlikely to collide again. Figure 3.2 shows a successful election after the leader (node 5) crashes.

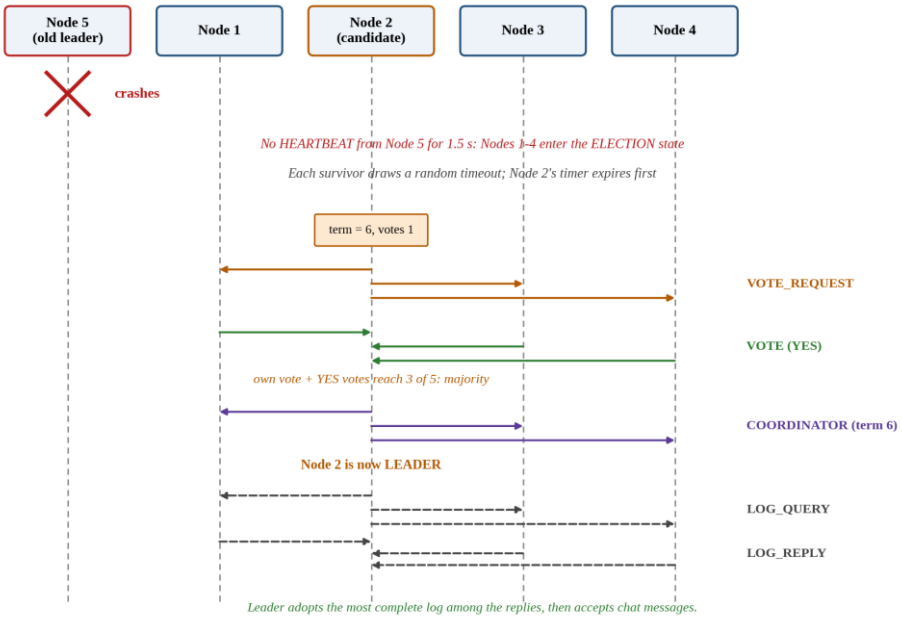



**Figure 3.2 Message sequence of an election after the leader (Node 5) crashes**

### *3.3.4 Bully and Ring elections*

The Bully and Ring algorithms are selected with a start-up option and share the rest of the system. In Bully, the node that detects the failure sends `ELECTION` to all higher-numbered nodes; any live higher node replies `ELECTION_OK` and starts its own election, and a node that hears no reply within 1.0 s declares itself leader with `COORDINATOR`. The highest live identifier therefore always wins. In Ring, the detecting node sends a `RING_ELECTION` token to its successor on the ring of live nodes; every node forwards the larger of the received identifier and its own, and the node that receives its own identifier back announces itself as leader.

### *3.3.5 Ordering, replication and commit*

A client sends `CHAT` with a unique message identifier. A follower forwards the message to the leader (`FORWARD`); the leader ignores an identifier it already holds, gives a new message the next sequence number, appends it to its log and sends `REPLICATE` to the followers together with the identifier of the preceding entry. A follower appends the entry only if its log matches up to that point, otherwise it asks for repair with `SYNC_REQ`. Followers answer with `ACK`; when a majority holds a sequence

number the leader commits it and broadcasts `COMMIT` (Figure 3.3). Each node then delivers committed entries, in order, to its own clients.

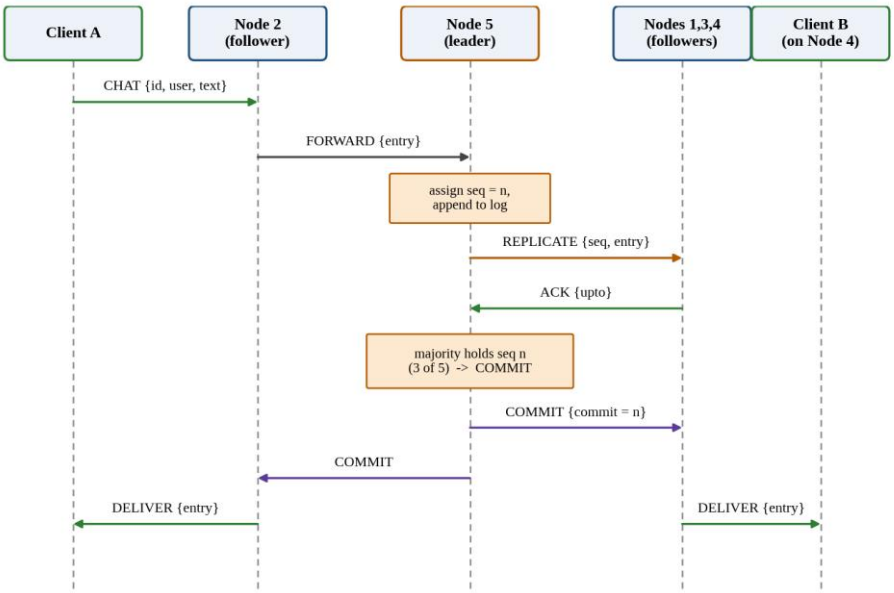



**Figure 3.3 Message sequence for one chat message (write path)**

Because there is one leader, one log and a majority rule, every client sees the same sequence of messages. A leader without a majority of reachable nodes cannot commit anything, which is how the system preserves consistency during a partition.

### *3.3.6 Leader catch-up and node recovery*

A newly elected leader does not serve writes immediately. It first sends `LOG_QUERY` to its peers, waits up to 1.0 s for `LOG_REPLY` messages and adopts the most complete log it receives, so that no committed message can be lost by electing a node that lagged behind. It then marks itself ready. A restarted node starts with an empty log; the first heartbeat shows that the leader's log is longer, so the node requests the missing entries with `SYNC_REQ` and the leader answers with `SYNC_RESP`. The same repair replaces an uncommitted suffix that diverged, for example on a former leader that was isolated.

### *3.3.7 Client failover*

A client sends `HELLO` with the sequence number it has last seen, so a node can resume delivery exactly after it. Nodes send `INFO` status messages periodically; a client that hears nothing for 3 s, or is told for more than 6 s that its node has no leader or no quorum, reconnects to another node. Messages that were not delivered back within 4 s are resent with the same identifier, and the leader ignores identifiers it already holds, so retries cannot create duplicates. The client also discards any delivered entry whose sequence number it has already seen.

### *3.3.8 Terms, quorum and fault injection*

Every election and every leader has a term number, and a message from an older term is ignored. A leader that was cut off therefore cannot overwrite the newer cluster state when it returns. For testing, each node's HTTP control endpoint can crash the node, isolate it from the network, add delay or packet loss, or make the leader resign; the cluster manager can restart a killed node. A "slow motion" option holds election messages in flight so that every vote can be watched on the dashboard; it is used only for demonstrations, never for measurements.

## 3.4 Execution Procedure

The project needs Python 3 (and matplotlib only to redraw the benchmark graph). From the project folder, the cluster, dashboard and tests are started as follows:

```
cd dchat
python dashboard.py --spawn           # 5 nodes + dashboard on port 8080
python client.py --user alice         # terminal chat client
python client.py --user bob --node 2    # second client, other node
python tests/scenario_test.py         # crash, restart, partition tests
python tests/winner_distribution.py --elections 20
python benchmark.py --sizes 3 5 7 9 --trials 3
```

The election algorithm is chosen with an option (`--algo vote`, `bully` or `ring`; voting is the default). Figure 3.4 shows the dashboard after start-up: node 1 is the leader of term 1, all five nodes hold the same ten committed messages, and the log fingerprints match.

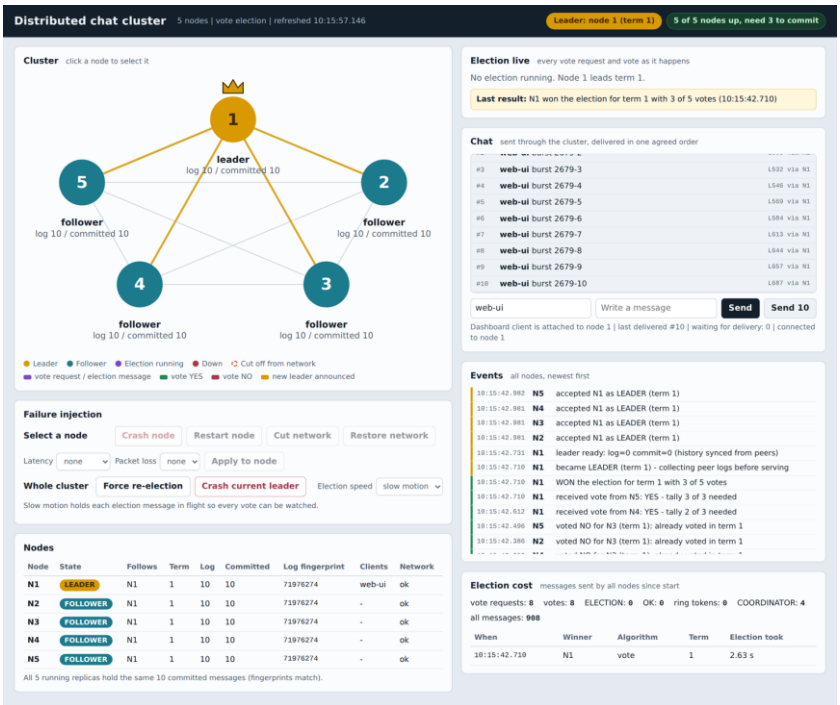



**Figure 3.4 Dashboard showing a healthy five-node cluster with Node 1 as leader**

# CHAPTER 4 : TESTING AND RESULTS

## 4.1 Test Plan

All tests ran on a single computer with every node, client and the dashboard on the loopback interface; the scenario tests use their own port range so they do not clash with a running dashboard. The scenario test starts five real node processes, attaches two clients to different nodes, injects faults with SIGKILL and network isolation, and checks the outcome automatically. A test passes only if every check inside it passes. The success criteria are: one agreed leader, a gapless and duplicate-free message stream identical on all clients, identical committed logs on all live nodes, and convergence after every fault.

Table 4.1 lists the tests and the results.

**Table 4.1 Test plan and results**

| ID | Test and input | Expected result | Observed result | Status |
|---|---|---|---|---|
| T1 | Start 5 nodes (voting) | All nodes agree on one ready leader | Leader = node 3 | Pass |
| T2 | Two clients on different nodes send 10 messages | Both receive all 10 in the same order, sequence gapless | 10 and 10 received; identical order | Pass |
| T3 | Kill the leader while clients keep typing | New leader; nothing lost, duplicated or reordered; live logs identical | Node 2 elected 2.6 s after the kill; 20 of 20 delivered | Pass |
| T4 | Restart the killed node with an empty log | It recovers all messages; one leader | 22 messages identical on every node; leader = node 2 | Pass |
| T5 | Isolate the current leader from the network, then heal | Majority elects another leader and keeps committing; logs converge | Node 3 elected on the majority side; identical logs after healing; last_seq = 25 | Pass |
| T6 | T1–T5 with Bully election | Same guarantees; restarted highest id takes over again | New leader node 4 in 1.6 s; node 5 regained leadership | Pass |
| T7 | T1–T5 with Ring election | Same guarantees | New leader node 4 in 1.5 s | Pass |
| T8 | 20 voting elections, count winners | Winners vary across nodes | Node 1: 0, node 2: 8, node 3: 1, node 4: 6, node 5: 5 | Pass, not uniform |
| T9 | Benchmark: 3 algorithms × 3, 5, 7, 9 nodes × 3 trials | Messages and recovery time recorded | 36 trials completed | Pass |
| T10 | Dashboard demo: force re-election, then crash leader during traffic | Cluster re-converges; fingerprints match | See Figures 4.1 to 4.3 | Pass |

**Not tested:** The following conditions were not tested: loss of a majority of nodes (the service is expected to stop committing), failure of all nodes at once, a run across several computers, and a quantitative measurement of the effect of injected delay and packet loss. The retry path that prevents duplicates is exercised by the failover test, but there is no dedicated test that forces a duplicate.

## 4.2 Results and Execution Evidence

### *4.2.1 Scenario test output*

The console output of the voting scenario test is shown below; it corresponds to tests T1 to T5. The Bully and Ring outputs are in Appendix B.

```
== 1. startup (vote, 5 nodes, round 1)
  PASS  all nodes agree on one ready leader  [leader = node 3]

== 2. normal chat with two clients on different nodes
  PASS  both clients receive all 10 messages  [alice=10 bob=10]
  PASS  identical total order on both clients
  PASS  sequence numbers are gapless

== 3. crash the leader (node 3) while clients keep typing
  PASS  a new leader is elected after the crash  [node 2, 2.6s after kill]
  PASS  no message lost across the failover (20 total on both clients)  [alice=20 bob=20]
  PASS  no duplicates
  PASS  same order on both clients
  PASS  every live node holds the same committed log

== 4. restart node 3 (empty log) - it must rejoin and recover everything
  PASS  22 messages committed on every node, identical logs
  PASS  one leader again  [node 2]
  PASS  clients received all 22

== 5. network partition of the current leader
  PASS  majority side elects a different leader  [node 3]
  PASS  majority keeps committing during the partition
  PASS  after healing, identical logs everywhere
  PASS  one leader after healing
  PASS  clients still see a gapless, duplicate-free stream  [last_seq=25]

ALL CHECKS PASSED
```

*Listing 4: output of python tests/scenario_test.py (voting election).*

### *4.2.2 Elections on the dashboard*

Figures 4.1 to 4.3 come from a live run of the dashboard in slow-motion mode, in which every election message is held in flight for a short time so that it can be watched; the durations on those screens are therefore longer than real-time values. Figure 4.1 was taken after the leader was made to resign. Four nodes (1, 3, 4 and 5) became candidates for term 2 at nearly the same time and each voted for itself, so each had one vote out of the three needed and the other nodes answered “already voted in term 2”. This is the split vote described in Section 3.3.3; the cluster did not stay stuck, because the cluster later settled on node 3 as leader of term 3.

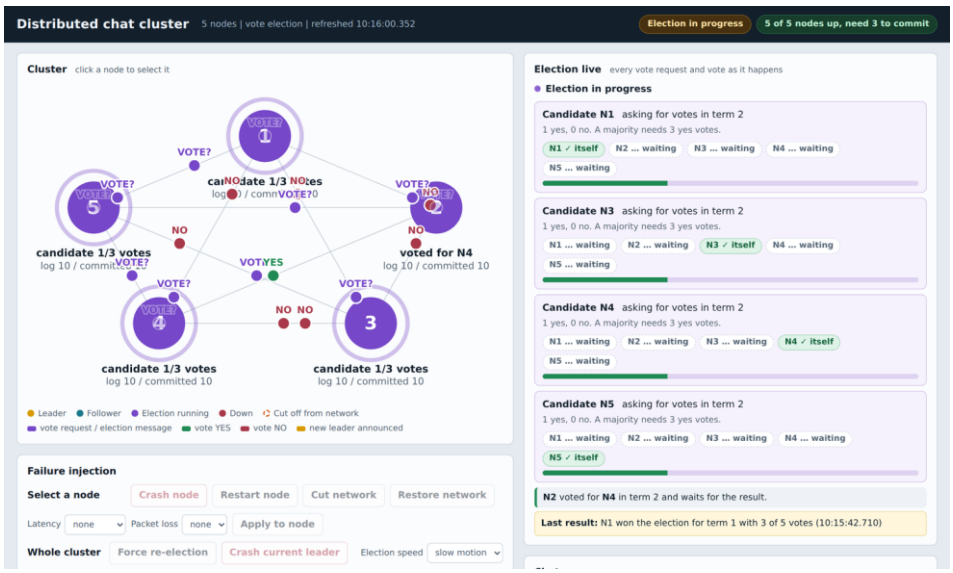



**Figure 4.1 Election in progress after a forced re-election (four candidates compete in term 2)**

Figure 4.2 was taken a few seconds after the leader (node 3) was crashed while a burst of messages was being sent. Node 3 is shown as down, the survivors have all entered the election state, and three of them are candidates for term 4. The chat panel shows that seven messages were still waiting to be delivered, and the nodes table shows the same committed log (13 entries, identical fingerprint) on all four live nodes.

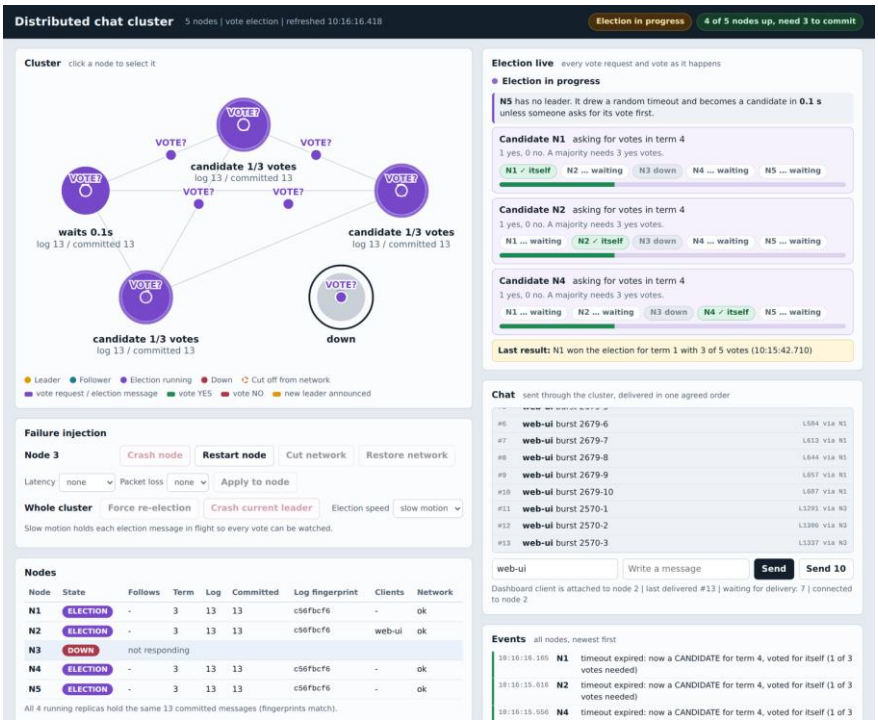



**Figure 4.2 Election after the leader (Node 3) was crashed during message traffic**

Figure 4.3 shows the outcome. Term 4 ended without a winner because of another split; node 4 then won term 5 with 3 of 5 votes, and the event log records “leader ready” after the new leader had synchronised its history from its peers. All 20 messages sent were delivered in order and the four live nodes report identical log fingerprints, so nothing was lost or duplicated across the failover.

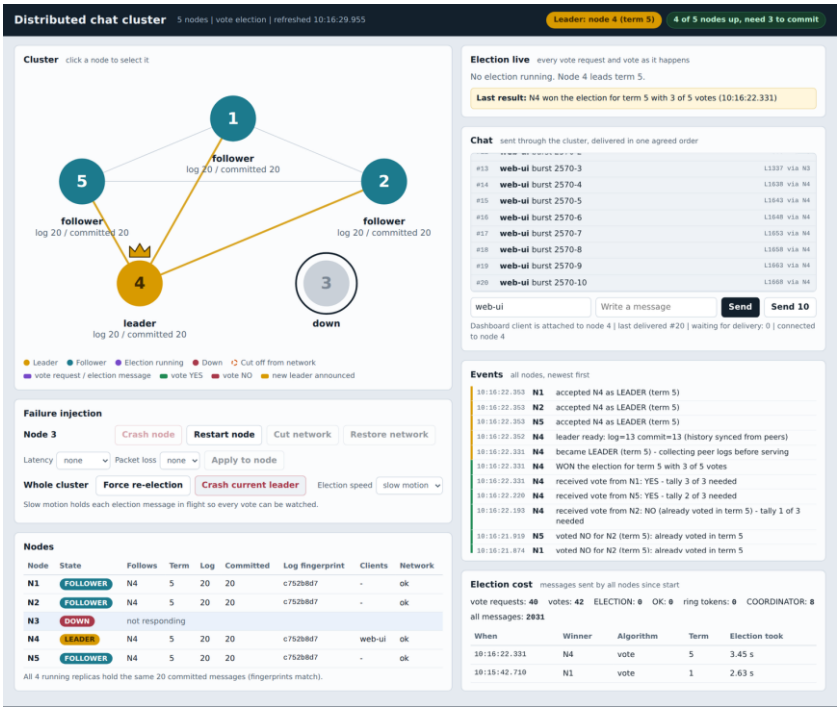



**Figure 4.3 Cluster after recovery: Node 4 leads term 5 and all live nodes hold identical logs**

### *4.2.3 Who wins an election*

Figure 4.4 compares the winners of 20 voting elections with those of 8 Bully elections on a five-node cluster. Under Bully node 5, the highest identifier, won every time, as the algorithm prescribes. Under voting, the winner changed from election to election and no node held the leadership permanently.

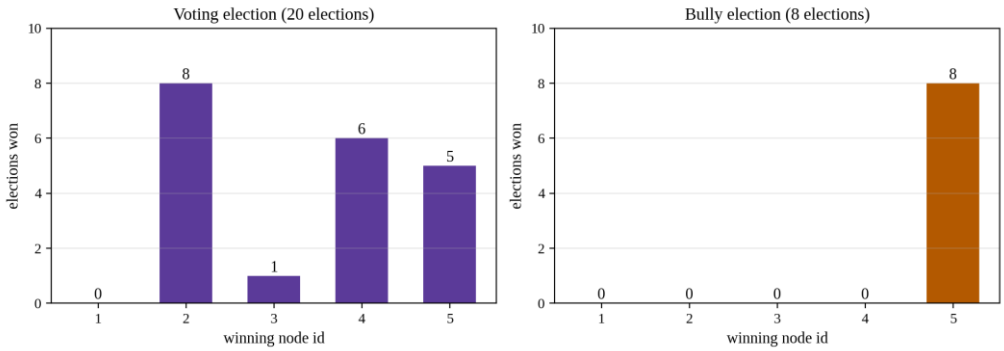



**Figure 4.4 Winners of repeated elections for voting and Bully**

The voting result is random but not uniform: node 1 never won, and node 2 won eight times, where four wins each would be expected. A chi-square test on these counts gives $\chi^2 = 11.5$ with 4 degrees of freedom, which would reject uniformity at the 5 % level. Consecutive elections are also not independent trials, so this should be read as an observation rather than a statistical proof. The cause was not investigated (Section 5.5).

### *4.2.4 Messages and recovery time*

The benchmark crashes the leader and measures, until all survivors agree on a new, synchronised leader, the detection time, the duration of the election itself, the total recovery time and the number

of election messages sent. Table 4.2 gives the mean of three trials per cluster size, taken from the project's `results/benchmark.csv`, and Figure 4.5 plots messages and recovery time against cluster size.

**Table 4.2 Mean benchmark results after a leader crash (3 trials per row)**

| Algorithm | Nodes | Detection (s) | Election phase (s) | Recovery (s), mean (min–max) | Election messages, mean (min–max) |
|---|---|---|---|---|---|
| Voting | 3 | 1.34 | 0.726 | 2.11 (1.70–2.34) | 5.0 (5–5) |
| Voting | 5 | 1.32 | 0.816 | 2.17 (1.81–2.54) | 11.0 (11–11) |
| Voting | 7 | 1.28 | 0.731 | 2.06 (1.76–2.29) | 17.0 (17–17) |
| Voting | 9 | 1.26 | 0.707 | 2.02 (1.81–2.23) | 23.0 (23–23) |
| Bully | 3 | 1.41 | 0.001 | 1.47 (1.40–1.61) | 6.7 (6–7) |
| Bully | 5 | 1.54 | 0.002 | 1.60 (1.60–1.61) | 20.0 (14–32) |
| Bully | 7 | 1.50 | 0.004 | 1.54 (1.43–1.60) | 38.3 (7–74) |
| Bully | 9 | 1.55 | 0.007 | 1.61 (1.58–1.63) | 70.0 (62–86) |
| Ring | 3 | 1.42 | 0.001 | 1.47 (1.40–1.61) | 4.7 (4–5) |
| Ring | 5 | 1.47 | 0.002 | 1.51 (1.38–1.60) | 10.0 (8–11) |
| Ring | 7 | 1.55 | 0.005 | 1.59 (1.59–1.59) | 16.3 (16–17) |
| Ring | 9 | 1.45 | 0.005 | 1.51 (1.45–1.63) | 18.3 (16–23) |

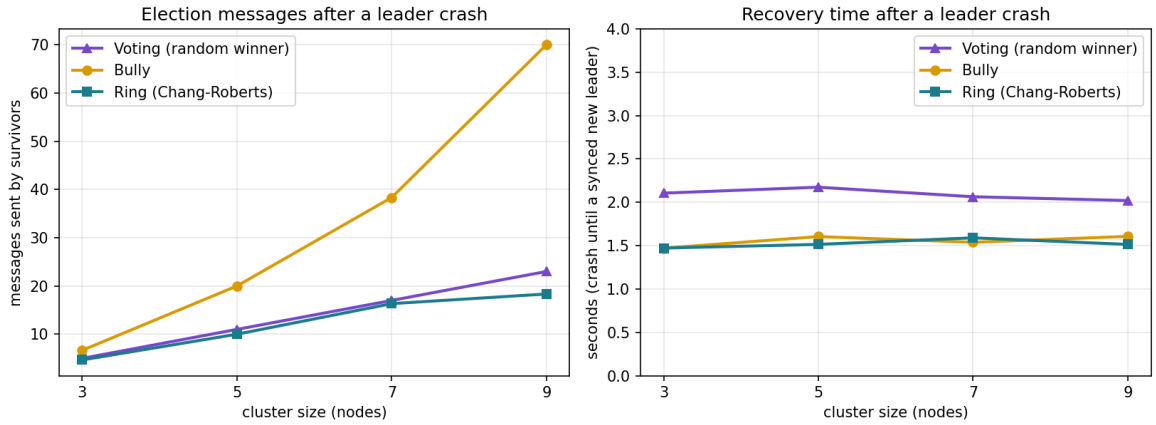



**Figure 4.5 Election messages and recovery time against cluster size (benchmark)**

Three observations follow. First, detection takes about 1.30 to 1.50 s for every algorithm, as expected from the 1.5 s failure timeout and 0.5 s heartbeat interval; the algorithms differ only afterwards. Second, the election phase of voting averages 0.74 s, because of the random wait of 0.5 to 1.5 s, whereas Bully and Ring finish in a few milliseconds on a local machine; overall recovery is therefore about 2.09 s for voting against 1.56 s for Bully and 1.52 s for Ring. The voting trials (1.70–2.54 s) lie clearly above the others (up to 1.63 s). Third, voting sent exactly $3n-4$ election messages in every trial (5, 11, 17 and 23 for 3, 5, 7 and 9 nodes), made up of $n-1$ vote requests, $n-2$ votes and $n-1$ announcements. Bully's message count is much higher and varies from trial to trial, reaching about 70 for nine nodes, because every node that notices the failure starts its own election.

### 4.3 Objective Achievement

**Table 4.3 Objective achievement**

| ID | Objective | Status | Evidence |
|---|---|---|---|
| O1 | Detect leader failure automatically | Achieved | T3; detection about 1.3–1.5 s |
| O2 | Elect exactly one leader by majority voting | Achieved; winner not uniform | T1, T5, T8; Fig. 4.1–4.4 |
| O3 | One total order, no loss or duplication | Achieved | T2, T3; Fig. 4.3 |
| O4 | Majority commit; restarted node recovers | Achieved | T4 |
| O5 | Consistent under partition | Achieved | T5 |
| O6 | Client failover and safe retry | Achieved | T3 (clients kept typing) |
| O7 | Compare voting, Bully and Ring | Achieved | Table 4.2, Fig. 4.5 |
| O8 | Live dashboard with fault injection | Achieved | Fig. 3.4, 4.1–4.3 |

# CHAPTER 5 : DISCUSSION AND CONCLUSION

## 5.1 Discussion and Comparative Analysis

The three election algorithms were run on the same system, so the differences in Table 5.1 come from the algorithms and not from the rest of the code. All three passed the same scenario checks.

**Table 5.1 Comparison of the three election algorithms (measured values)**

| Criterion | Voting | Bully | Ring |
|---|---|---|---|
| Who wins | Any live node (random timers) | Highest live id | Highest live id |
| Election messages, 5 nodes | 11.0 | 20.0 | 10.0 |
| Election messages, 9 nodes | 23.0 | 70.0 | 18.3 |
| Mean recovery, all sizes | 2.09 s | 1.56 s | 1.52 s |
| Split votes possible | Yes, resolved by retry | No | No |
| Restarted node with highest id | Follows the current leader | Takes leadership back | Takes leadership back |

Voting recovers about half a second more slowly because of its random wait, but it spreads leadership across the nodes, uses a number of messages that grows linearly with the cluster size, and does not disturb a working leader when a node with a higher identifier restarts. Bully is the fastest to settle on a local machine but always loads the highest-numbered node, sends many more messages as the cluster grows, and replaces a healthy leader whenever a higher id returns. Ring was as fast as Bully here and used the fewest messages at nine nodes; its cost, not measured here, is that the token must travel around the whole ring.

These conclusions have limits. Each point is the mean of only three trials, all nodes shared one computer so the network added almost no delay, and the differences between Bully and Ring are within the variation between trials. Only the gap between voting and the other two is clearly larger than that variation. The measured winners of voting were also not uniform (Section 4.2.3).

The system favours consistency over availability, in the sense of the CAP result [7]. Messages are delivered only after a majority has stored them, so a node that is cut off from the majority stops accepting writes instead of creating a second history. This is the right choice for a chat log whose order must be identical for everybody, but it means a minority partition is unavailable until the network heals.

## 5.2 Advantages and Applications

- **Automatic recovery:** a crashed leader is replaced in about two seconds without operator action, and a restarted node recovers its history on its own.

- **One agreed order:** every client sees the same sequence of messages, with no loss and no duplicates across failovers.
- **No leader favouritism:** the voting election does not tie leadership to a node's identifier.
- **Easy to run and inspect:** only the Python standard library is needed, and the dashboard shows every vote, message count and log fingerprint.
- **Selectable algorithm:** voting, Bully and Ring can be compared on the same code base.

The ideas are the same as those used in real replicated services: coordination services, replicated databases and message brokers elect a leader and replicate an ordered log by majority [5]. The project is also useful as a teaching tool for failure detection, election and replication in a networks laboratory.

### 5.3 Limitations

- The message log is kept in memory only; if all nodes stop, the history is lost.
- The set of nodes is fixed at start-up; nodes cannot be added or removed while the cluster runs.
- There is no encryption or authentication; any process that can reach a port can join as a client.
- The replication protocol is a simplified, Raft-inspired design whose safety was checked by tests but not formally proved.
- All measurements were made on one computer over the loopback interface, with only three trials per configuration and cluster sizes from 3 to 9 nodes. Real network delay, and a run on several computers, were not evaluated.
- Loss of a majority of nodes, failure of all nodes at once, and quantitative effects of delay and packet loss were not tested (Section 4.1).
- The winners of the voting election were not uniformly distributed in the test, and the reason is not yet understood.

### 5.4 Challenges and Troubleshooting

**Table 5.2 Issues, causes and solutions**

| Issue | Cause | Solution and verification |
|---|---|---|
| Split vote: nobody reaches a majority | Several nodes' random timers expire at nearly the same moment | Random wait of 0.5–1.5 s; a failed round is repeated with a fresh random timeout. Seen in Fig. 4.1 and 4.3 and resolved in a later term. |
| Old leader returns after a partition | An isolated leader still believes it is in charge | Term numbers: messages from an older term are ignored and the old leader steps down. Test T5. |
| Duplicate messages after a client retries | A message resent after failover may already be in the log | Unique message identifiers: the leader ignores ids it already holds and clients drop sequence numbers already seen. Test T3. |

| Issue | Cause | Solution and verification |
|---|---|---|
| A new leader that lacks committed history | The elected node could have fallen behind | Votes are refused for a candidate whose log is behind; the new leader collects peers' logs before serving. Tests T3, T4. |
| Stale traffic replayed when a dead peer returns | Messages queued for an unreachable peer would arrive late | The outbound link discards queued messages while a peer is unreachable, as noted in the source code. |

### 5.5 Future Scope

- Store the log on disk so that history survives a full restart.
- Support adding and removing nodes while the cluster is running.
- Add TLS and user authentication.
- Follow the full Raft log-replication rules and check the protocol with a model checker.
- Repeat the benchmark with more trials and on several computers over a real network, and investigate why some nodes win voting elections more often than others.
- Adapt the failure timeout to measured network delay instead of using a fixed 1.5 s.
- Add chat rooms, history compaction and message batching for larger deployments.

### 5.6 Conclusion

This project built a distributed chat service in which several Python server nodes detect a failed leader with heartbeats, elect a new one by majority voting (or by the Bully or Ring algorithm), and keep one agreed, majority-committed message order. Scenario tests on a five-node cluster showed that no message was lost, duplicated or reordered when the leader was killed during traffic or isolated by a partition, and that a restarted node recovered the full history. The benchmark showed that voting recovers in about 2.1 s, roughly half a second slower than Bully and Ring, and uses $3n-4$ election messages, whereas Bully's message count grows much faster.

The stated objectives were met within the limits of a single-computer evaluation. The main lessons are that failure detection must tolerate a late heartbeat, that randomised timeouts make split votes rare but not impossible, and that safety under partitions comes from combining terms with a majority rule. The most valuable next steps are persistence, a multi-computer evaluation and a formal check of the protocol.

# REFERENCES


[1] A. S. Tanenbaum and M. van Steen, *Distributed Systems: Principles and Paradigms*, 2nd ed. Upper Saddle River, NJ, USA: Pearson Prentice Hall, 2007.

[2] W. Eddy, Ed., "Transmission Control Protocol (TCP)," Internet Engineering Task Force, RFC 9293, Aug. 2022. [Online]. Available: https://www.rfc-editor.org/info/rfc9293.

[3] H. Garcia-Molina, "Elections in a distributed computing system," *IEEE Trans. Comput.*, vol. C-31, no. 1, pp. 48–59, Jan. 1982.

[4] E. Chang and R. Roberts, "An improved algorithm for decentralized extrema-finding in circular configurations of processes," *Commun. ACM*, vol. 22, no. 5, pp. 281–283, May 1979.

[5] D. Ongaro and J. Ousterhout, "In search of an understandable consensus algorithm," in *Proc. USENIX Annu. Tech. Conf. (USENIX ATC)*, Philadelphia, PA, USA, Jun. 2014, pp. 305–319.

[6] L. Lamport, "Time, clocks, and the ordering of events in a distributed system," *Commun. ACM*, vol. 21, no. 7, pp. 558–565, Jul. 1978.

[7] S. Gilbert and N. Lynch, "Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services," *ACM SIGACT News*, vol. 33, no. 2, pp. 51–59, Jun. 2002.

[8] Python Software Foundation, "socket — Low-level networking interface," *Python 3 Documentation*. [Online]. Available: https://docs.python.org/3/library/socket.html